

Gradient descent is a method for unconstrained [mathematical optimization](#). It is a [first-order iterative algorithm](#) for minimizing a [differentiable multivariate function](#).

The idea is to take repeated steps in the opposite direction of the [gradient](#) (or approximate gradient) of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as *gradient ascent*. It is particularly useful in [machine learning](#) for minimizing the cost or loss function.^[1] Gradient descent should not be confused with [local search](#) algorithms, although both are [iterative methods](#) for [optimization](#).

Gradient descent is generally attributed to [Augustin-Louis Cauchy](#), who first suggested it in 1847.^[2] [Jacques Hadamard](#) independently proposed a similar method in 1907.^{[3][4]} Its convergence properties for non-linear optimization problems were first studied by [Haskell Curry](#) in 1944,^[5] with the method becoming increasingly well-studied and used in the following decades.^{[6][7]}

A simple extension of gradient descent, [stochastic gradient descent](#), serves as the most basic algorithm used for training most [deep networks](#) today.

Description

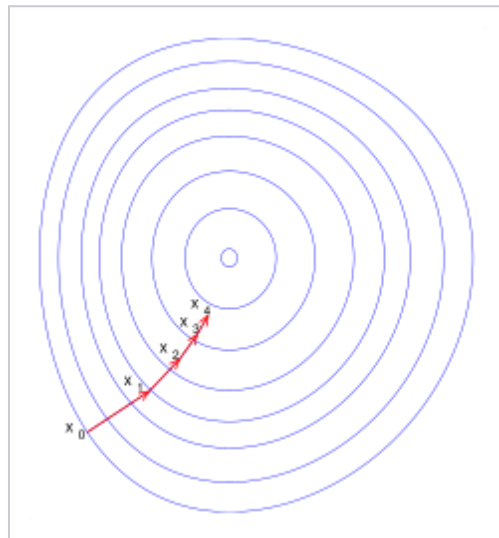


Illustration of gradient descent on a series of [level sets](#)

Gradient descent is based on the observation that if the [multi-variable function](#)

$f(x)$

is **defined** and **differentiable** in a neighborhood of a point

a

, then

$f(x)$

decreases *fastest* if one goes from

a

in the direction of the negative **gradient** of

f

at

$a, -\nabla f(a)$

. It follows that, if

$$a_{n+1} = a_n - \eta \nabla f(a_n)$$

for a small enough step size or **learning rate**

$\eta \in \mathbb{R}^+$

, then

$f(a_n) \geq f(a_{n+1})$

. In other words, the term

$\eta \nabla f(a)$

is subtracted from

a

☐ because we want to move against the gradient, toward the local minimum. With this observation in mind, one starts with a guess

x_0

☐ for a local minimum of

f

☐, and considers the sequence

$x_0, x_1, x_2, \dots$

such that

$$x_{n+1} = x_n - \eta_n \nabla f(x_n), \quad n \geq 0.$$

We have a **monotonic** sequence

$$f(x_0) \geq f(x_1) \geq f(x_2) \geq \dots,$$

so the sequence

(x_n)

☐ converges to the desired local minimum. Note that the value of the *step size*

η

☐ is allowed to change at every iteration.

It is possible to guarantee the **convergence** to a local minimum under certain assumptions on the function

f

☐ (for example,

f

☐ [convex](#) and

∇f

☐ [Lipschitz](#)) and particular choices of

η

☐. Those include the sequence

$$\eta_n = \frac{1}{\| \nabla f(x_n) - \nabla f(x_{n-1}) \|^2}$$



as in the [Barzilai-Borwein method](#),^{[8][9]} or a sequence

η_n

☐ satisfying the [Wolfe conditions](#) (which can be found by using [line search](#)). When the function

f

☐ is [convex](#), all local minima are also global minima, so in this case gradient descent can converge to the global solution.

This process is illustrated in the adjacent picture. Here,

f

☐ is assumed to be defined on the plane, and that its graph has a [bowl](#) shape. The blue curves are the [contour lines](#), that is, the regions on which the value of

f

 is constant. A red arrow originating at a point shows the direction of the negative gradient at that point. Note that the (negative) gradient at a point is **orthogonal** to the contour line going through that point. We see that gradient *descent* leads us to the bottom of the bowl, that is, to the point where the value of the function

f

 is minimal.

An analogy for understanding gradient descent



Fog in the mountains

The basic intuition behind gradient descent can be illustrated by a hypothetical scenario. People are stuck in the mountains and are trying to get down (i.e., trying to find the global minimum). There is heavy fog such that visibility is extremely low. Therefore, the path down the mountain is not visible, so they must use local information to find the minimum. They can use the method of gradient descent, which involves looking at the steepness of the hill at their current position, then proceeding in the direction with the steepest descent (i.e., downhill). If they were trying to find the top of the mountain (i.e., the maximum), then they would proceed in the direction of steepest ascent (i.e., uphill). Using this method, they would eventually find their way down the mountain or possibly get stuck in some hole (i.e., local minimum or **saddle point**), like a mountain lake. However, assume also that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a sophisticated instrument to measure, which the people happen to have at that moment. It takes quite some time to measure the steepness of the hill with the instrument. Thus, they should minimize their use of the instrument if they want to get down the mountain before sunset. The difficulty then is

choosing the frequency at which they should measure the steepness of the hill so as not to go off track.

In this analogy, the people represent the algorithm, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore. The steepness of the hill represents the [slope](#) of the function at that point. The instrument used to measure steepness is [differentiation](#). The direction they choose to travel in aligns with the [gradient](#) of the function at that point. The amount of time they travel before taking another measurement is the step size.

Activation Functions:

In [artificial neural networks](#), the activation function of a node is a function that calculates the output of the node based on its individual inputs and their weights. Nontrivial problems can be solved using only a few nodes if the activation function is *nonlinear*.^[1]

Modern activation functions include the logistic ([sigmoid](#)) function used in the 2012 [speech recognition](#) model developed by [Hinton](#) et al.,^[2] the [ReLU](#) used in the 2012 [AlexNet](#) computer vision model^{[3][4]} and in the 2015 [ResNet](#) model; and the smooth version of the ReLU, the [GELU](#), which was used in the 2018 [BERT](#) model.^[5]

Comparison of activation functions

Aside from their empirical performance, activation functions also have different mathematical properties:

Nonlinear

When the activation function is non-linear, then a two-layer neural network can be proven to be a universal function approximator.^[6] This is known as the [Universal Approximation Theorem](#). The identity activation function does not satisfy this property. When multiple layers use the identity activation function, the entire network is equivalent to a single-layer model.

Range

When the range of the activation function is finite, gradient-based training methods tend to be more stable, because pattern presentations significantly affect only limited weights. When the range is infinite, training is generally more efficient because

pattern presentations significantly affect most of the weights. In the latter case, smaller [learning rates](#) are typically necessary.^{[[citation needed](#)]}

Continuously differentiable

This property is desirable for enabling gradient-based optimization methods ([ReLU](#) is not continuously differentiable and has some issues with gradient-based optimization, but it is still possible). The binary step activation function is not differentiable at 0, and it differentiates to 0 for all other values, so gradient-based methods can make no progress with it.^[7]

These properties do not decisively influence performance, nor are they the only mathematical properties that may be useful. For instance, the strictly positive range of the [softplus](#) makes it suitable for predicting variances in [variational autoencoders](#).